Industrial Internship Report on

” Smart City Traffic Pattern Forecasting”

Prepared by

Nikita Kishanlal Mali

Silver oak college of engineering and technology

Computer Engineering

Executive Summary

This report presents the work completed during the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with UniConverge Technologies Pvt. Ltd. (UCT).

The project, titled **Smart City Traffic Pattern Forecasting**, focuses on predicting vehicle counts at different traffic junctions using historical traffic data and machine learning techniques.

The project involved data preprocessing, exploratory data analysis, feature engineering, model development, and performance evaluation. A baseline Random Forest Regressor was developed and improved using historical traffic features such as lag values and rolling averages.

The improved Random Forest model achieved an MAE of 3.1206, an RMSE of 5.0047, and an R^2 score of 0.9662 on the validation dataset. It was also used to generate 11,808 predictions for the unseen test dataset across four traffic junctions.

This internship provided practical experience in data analysis, machine learning implementation, model evaluation, and solving a real-world traffic forecasting problem.

TABLE OF CONTENTS

1 Preface....3

2 Introduction....4

2.1 About UniConverge Technologies Pvt Ltd....4

2.2 About upskill Campus....8

2.3 The IOT Academy....9

2.4 Objective....9

2.5 Reference....9

2.6 Glossary....10

3 Problem Statement....10

4 Existing and Proposed solution....11

4.1 Existing Solution....11

4.2 Proposed Solution....12

5 Proposed Design/ Model....13

5.1 High Level Diagram....13

5.2 Low Level Diagram....14-17

5.3 Interfaces....18-22

6 Performance Test....23

6.1 Test Plan/ Test Cases....23

6.2 Test Procedure....23

6.3 Performance Outcome....24-27

7 My learnings....28

8 Future work scope....29

1 Preface

The six-week Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with UniConverge Technologies Pvt. Ltd. (UCT) was a valuable learning experience.

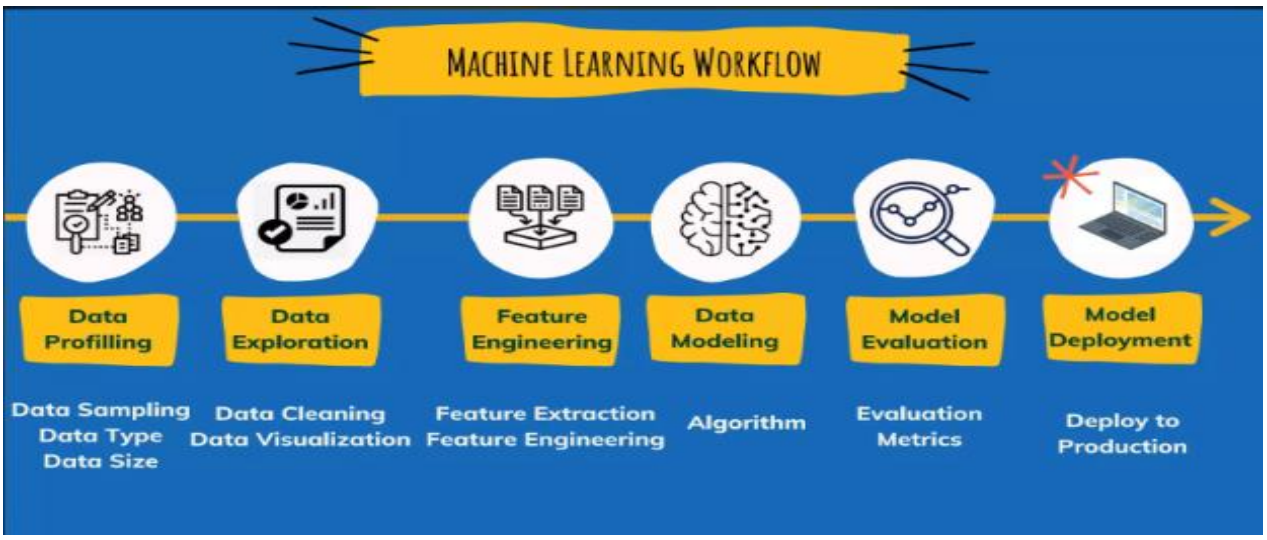
During this internship, I worked on **Smart City Traffic Pattern Forecasting**, a machine learning project focused on predicting vehicle counts at different traffic junctions using historical traffic data.

The project involved data preprocessing, exploratory data analysis, feature engineering, Random Forest model development, performance evaluation, and test dataset prediction.

The internship provided practical exposure to real-world problem-solving and helped me improve my technical knowledge, analytical thinking, and machine learning skills.

I sincerely thank upskill Campus, The IoT Academy, UCT, and everyone who supported me throughout this internship.

I encourage my juniors and peers to participate in such programs to gain practical experience and develop industry-relevant skills.



2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI.

For developing its products and solutions it is leveraging various **Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end** etc.



i. UCT IoT Platform (uct Insight)

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable "insight" for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to

• Build Your own dashboard

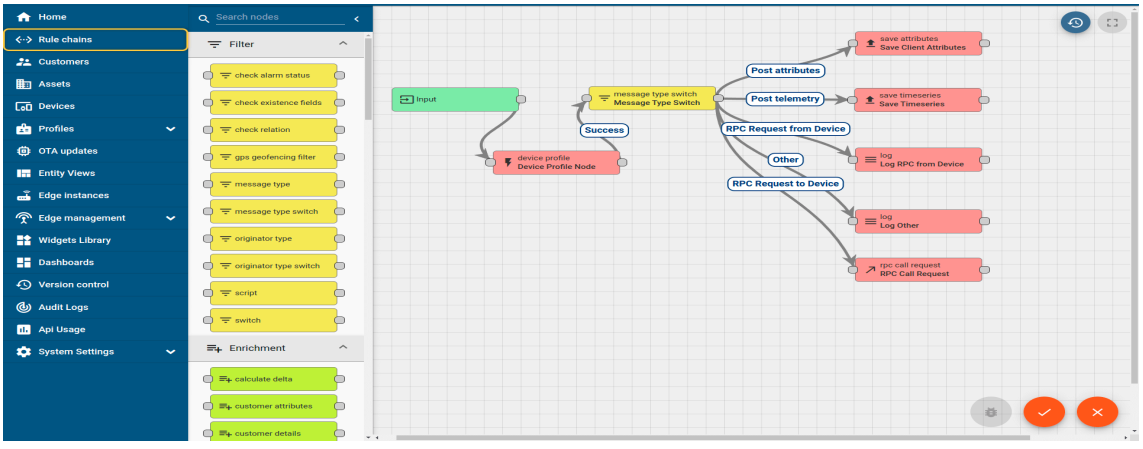
• Analytics and Reporting

• Alert and Notification

• Integration with third party application(Power BI, SAP, ERP)

• Rule Engine





ii. Smart Factory Platform (FACTORY WATCH)

Factory watch is a platform for smart factory needs.

It provides Users/ Factory

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.



Machine	Operator	Work Order ID	Job ID	Job Performance	Job Progress		Output		Rejection	Time (mins)				Job Status	End Customer
					Start Time	End Time	Planned	Actual		Setup	Pred	Downtime	Idle		
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i
CNC_S7_81	Operator 1	WO0405200001	4168	58%	10:30 AM		55	41	0	80	215	0	45	In Progress	i

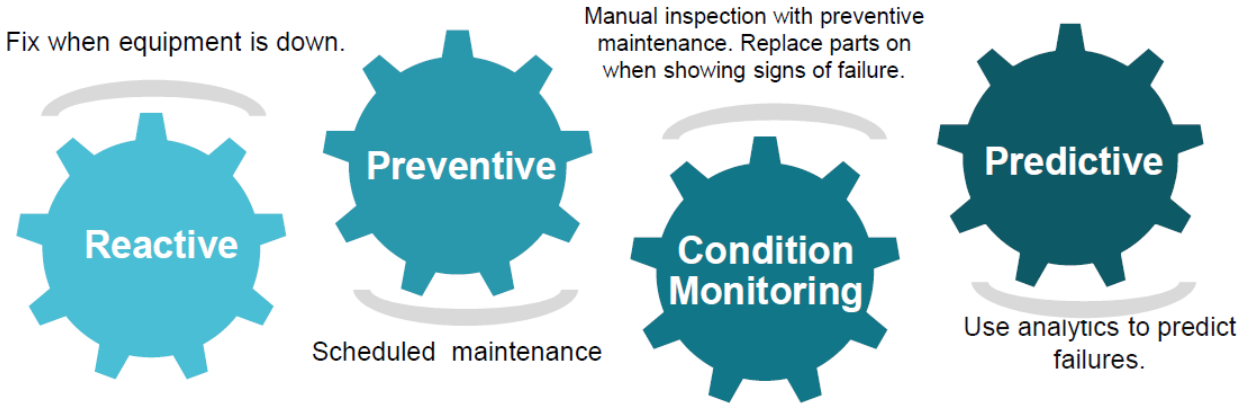


iii. LoRaWAN based Solution

UCT is one of the early adopters of LoRAWAN teschnology and providing solution in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

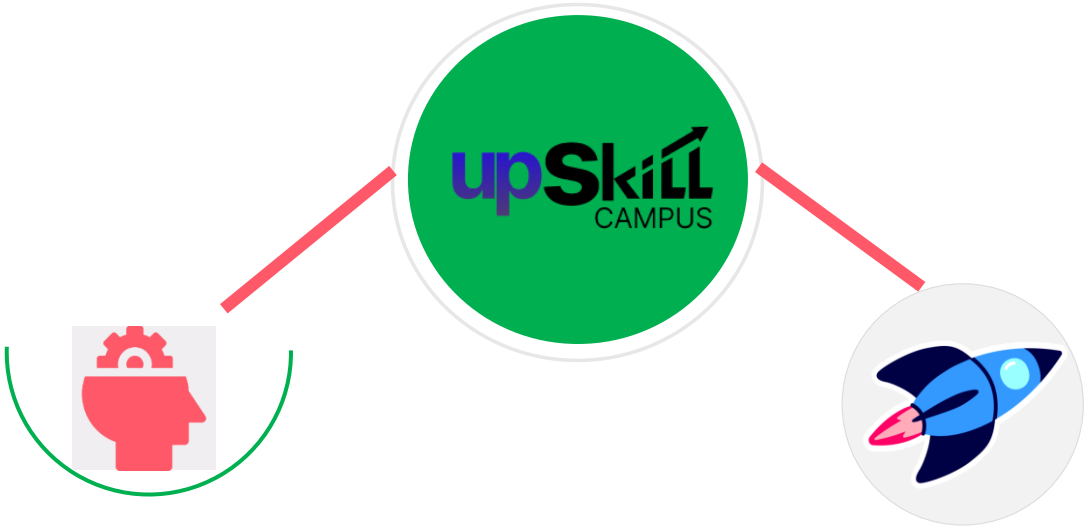
UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.



2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

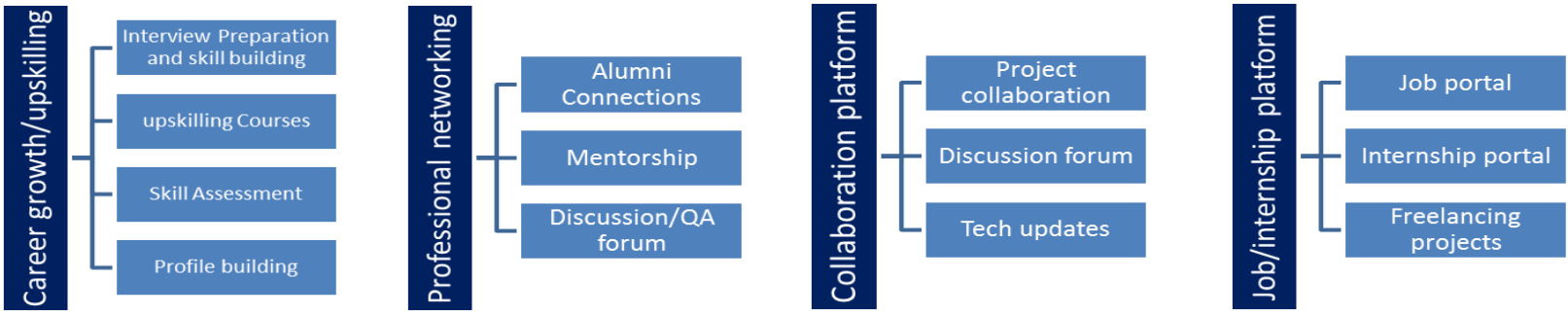
USC is a career development platform that delivers **personalized executive coaching** in a more affordable, scalable and measurable way.



Seeing need of upskilling in self paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services

upSkill Campus aiming to upskill 1 million learners in next 5 year

https://www.upskillcampus.com/



2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The main objectives of this internship program were:

- ☛ To gain practical experience in solving industry-related problems.
- ☛ To understand the application of machine learning in real-world scenarios.
- ☛ To develop skills in data preprocessing, analysis, and visualization.
- ☛ To implement and evaluate machine learning algorithms.
- ☛ To improve technical knowledge, analytical thinking, and problem-solving abilities.
- ☛ To gain experience in project development and documentation.

2.5 Reference

[1] Scikit-learn Documentation – Random Forest Regressor. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html

[2] Pandas Documentation. https://pandas.pydata.org/docs/

[3] NumPy Documentation. https://numpy.org/doc/

[4] Matplotlib Documentation. https://matplotlib.org/stable/

2.6 Glossary

Term	Acronym / Meaning
Machine Learning	ML
Exploratory Data Analysis	EDA
Random Forest	RF
Mean Absolute Error	MAE
Root Mean Squared Error	RMSE
Coefficient of Determination	R^2
Application Programming Interface	API
Lag Feature	Previous observation value
Rolling Mean	Moving average

3 Problem Statement

With the increasing number of vehicles in smart cities, traffic congestion has become a major challenge. Heavy traffic at road junctions leads to delays, increased fuel consumption, and inefficient transportation management.

Traditional traffic management systems often rely on fixed timings or manual monitoring, which may not accurately reflect changing traffic conditions.

The objective of this project, **Smart City Traffic Pattern Forecasting**, is to develop a machine learning model that predicts the number of vehicles at different traffic junctions using historical traffic data.

The project aims to:

1. Analyze historical vehicle count data from multiple traffic junctions.
2. Identify traffic patterns based on date and time.
3. Develop a machine learning model to predict vehicle counts.
4. Evaluate the model using suitable performance metrics.
5. Generate predictions for unseen traffic data to support better traffic planning.

4 Existing and Proposed solution

4.1 Existing Solution

Traditional traffic management systems commonly use fixed-time traffic signals and manual traffic monitoring. These methods may not adapt effectively to changing traffic conditions throughout the day.

Such systems have limitations:

- **They may not consider historical traffic patterns.**
- **Fixed signal timings may not reflect actual vehicle density.**
- **Manual monitoring requires continuous human involvement.**
- **They provide limited support for predicting future traffic volume**

4.2 Proposed Solution

The proposed system uses machine learning to predict vehicle counts at different traffic junctions based on historical traffic data.

A Random Forest Regressor was implemented and improved using time-based features and historical vehicle count features, including lag values and rolling averages.

The proposed approach includes:

1. **Collecting and analyzing historical traffic data.**
2. **Preprocessing the dataset and extracting time-based features.**
3. **Creating lag and rolling average features.**
4. **Training and evaluating the Random Forest model.**
5. **Generating vehicle count predictions for unseen test data.**

4.3 Code Submission (GitHub Link)

GitHub Repository: ***https://github.com/nikitamali21/upskillCampus/blob/main/Project-09-Smart-City-Traffic-Forecasting/Project9_Traffic_Forecasting.ipynb***

4.4 Report Submission (GitHub Link)

Project Report: ***https://github.com/nikitamali21/upskillCampus/blob/main/Project-09-Smart-City-Traffic-Forecasting/SmartCityTrafficForecasting_Nikita_USC_UCT.pdf***

5 Proposed Design/ Model

The proposed system uses a Random Forest Regression model to forecast vehicle counts at four traffic junctions. Historical traffic data is processed to extract useful time-based and traffic-related features before training the model.

5.1 High Level Diagram

The overall workflow of the proposed system is represented in the following diagram.



Figure5. 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

5.2 Low Level Diagram

The detailed implementation consists of the following steps:

1. **Data Loading:** Load the training and testing datasets using Pandas.
2. **Data Preprocessing:** Convert date-time values into the required format and check the dataset for inconsistencies.
3. **Feature Engineering:** Extract Hour, Day, Month, DayOfWeek, IsWeekend, and Year. Generate Lag_1, Lag_2, and Rolling_Mean_3 features.
4. **Model Training:** Train a Random Forest Regressor using the selected features.
5. **Model Evaluation:** Evaluate the model using MAE, RMSE, and R^2 score.
6. **Prediction:** Generate vehicle count predictions for the unseen test dataset.

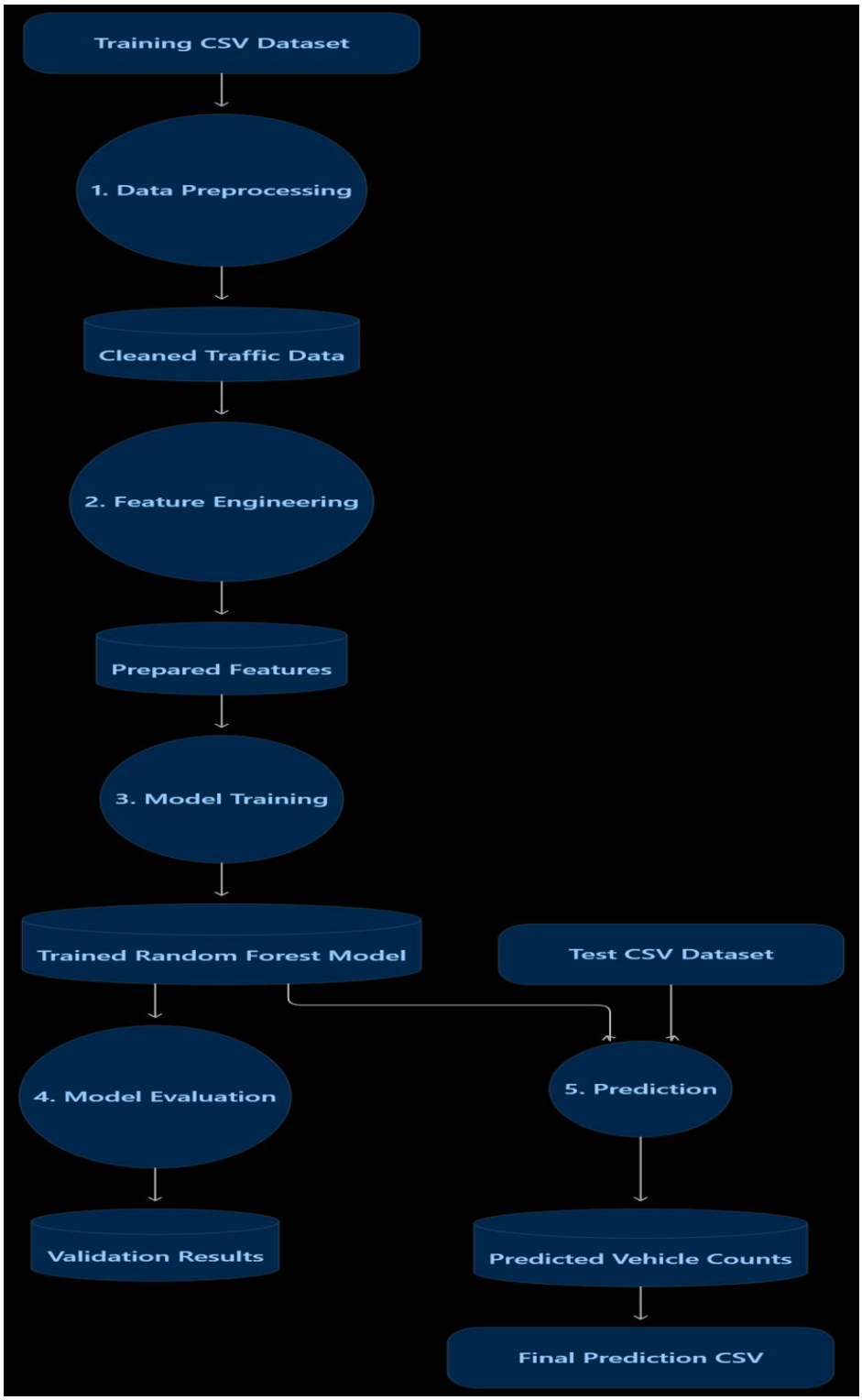


Figure 5.2: Data Flow Diagram (DFD)

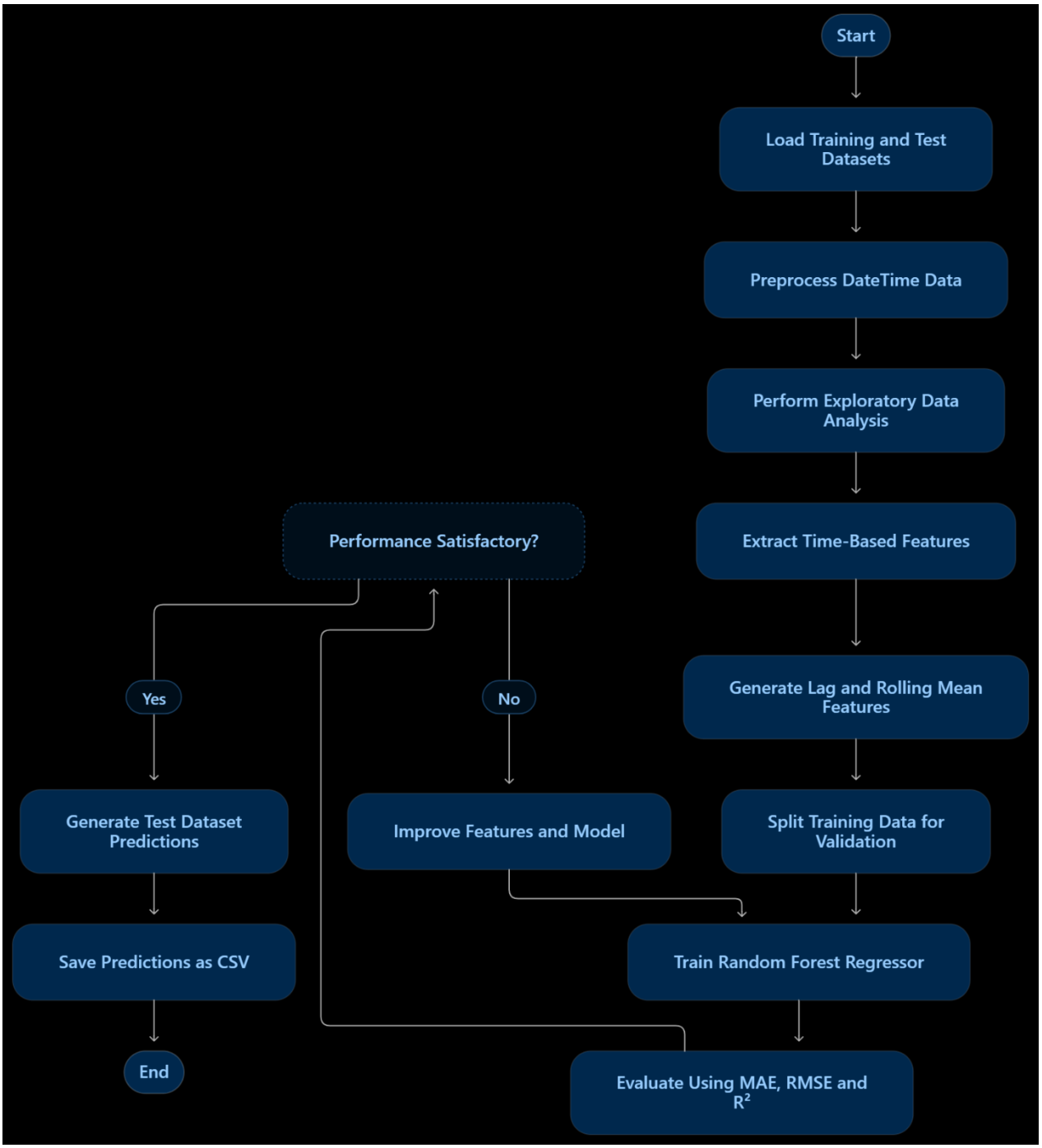


Figure 5.3: Flowchart

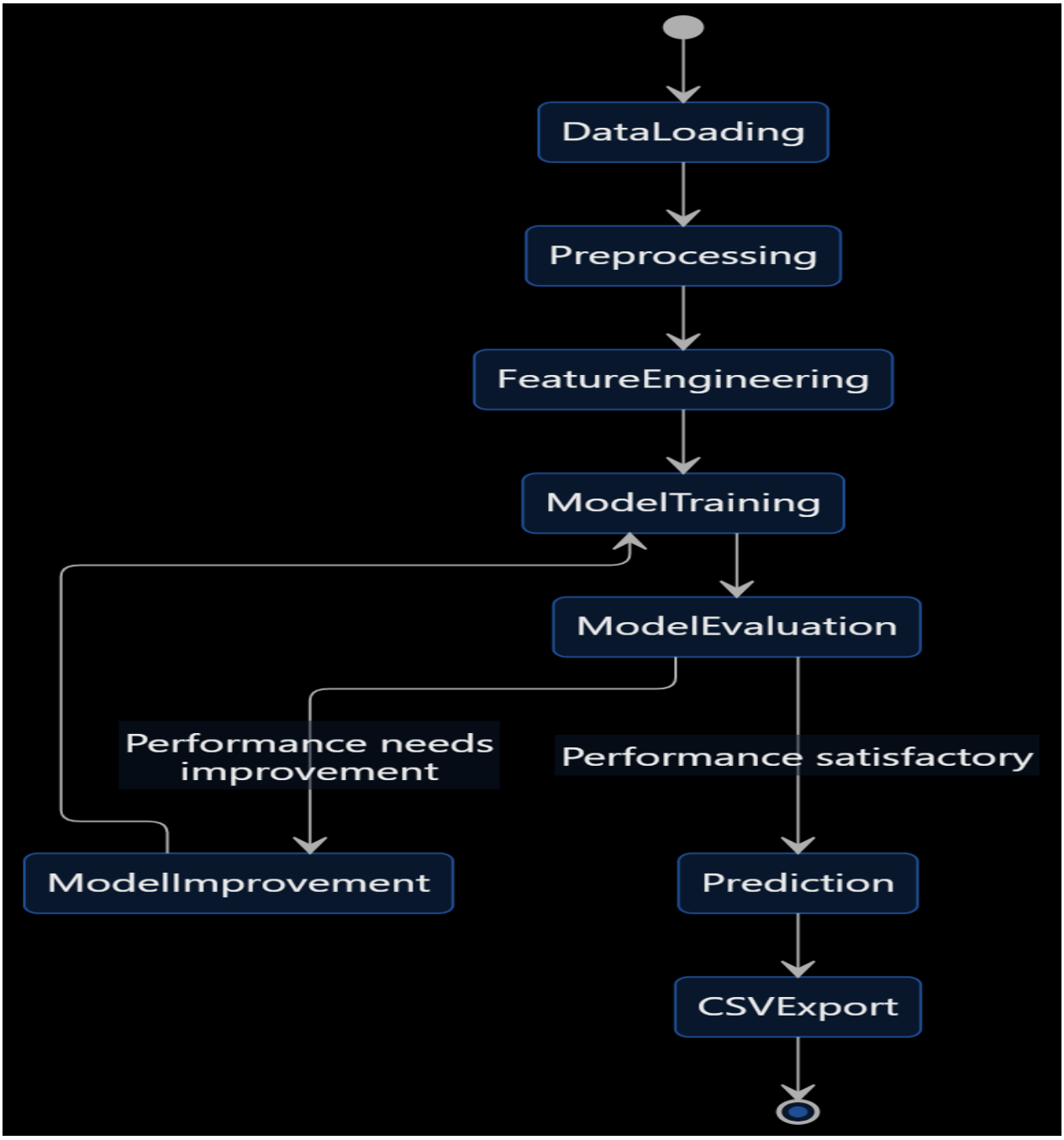


Figure 5.4: State Machine

5.3 Interfaces (if applicable)

The project was implemented using Python in a notebook-based environment. The main interfaces and tools used were:

- **Google Colab / Jupyter Notebook:** For writing and executing Python code.
- **Pandas and NumPy:** For data processing and numerical operations.
- **Matplotlib:** For visualizing traffic patterns and model results.
- **Scikit-learn:** For model training and performance evaluation.
- **CSV Files:** For storing the input datasets and final vehicle count predictions.

```python
import pandas as pd
import numpy as np

# Load training data
train = pd.read_csv("/content/train_aWnotuB.csv")

print("Dataset Shape:", train.shape)

print("\nColumn Names:")
print(train.columns.tolist())

print("\nData Types:")
print(train.dtypes)

print("\nFirst 10 Rows:")
display(train.head(10))

print("\nMissing Values:")
print(train.isnull().sum())

print("\nDuplicate Rows:")
print(train.duplicated().sum())

print("\nBasic Statistics:")
display(train.describe())
```

Dataset loading and preprocessing.

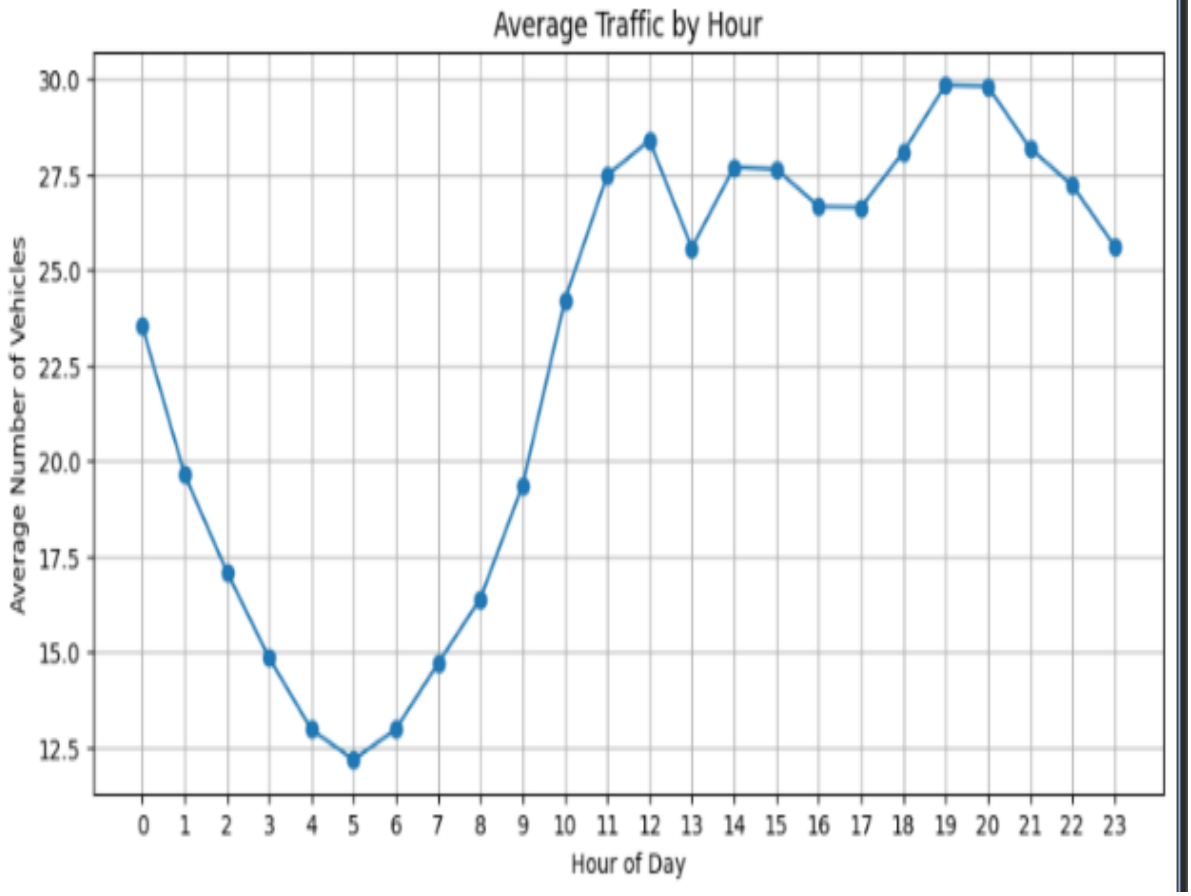
```
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

plt.plot(hourly_traffic.index, hourly_traffic.values, marker="o")

plt.title("Average Traffic by Hour")
plt.xlabel("Hour of Day")
plt.ylabel("Average Number of Vehicles")
plt.xticks(range(24))
plt.grid(True)

plt.show()
```



Exploratory data analysis and visualizations.

```python
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

# Create the Random Forest model
rf_model = RandomForestRegressor(
    n_estimators=100,
    random_state=42,
    n_jobs=-1
)

# Train the model
rf_model.fit(X_train, y_train)

# Make predictions
y_pred = rf_model.predict(X_val)

# Evaluate the model
mae = mean_absolute_error(y_val, y_pred)
rmse = np.sqrt(mean_squared_error(y_val, y_pred))
r2 = r2_score(y_val, y_pred)

print("Baseline Random Forest Results")
print("--------------------------------")
print("MAE :", round(mae, 4))
print("RMSE:", round(rmse, 4))
print("R²  :", round(r2, 4))
```

```
Baseline Random Forest Results
--------------------------------
MAE : 5.352
RMSE: 9.2945
R²  : 0.8834
```

Random Forest model training.

```python
# Create a copy of the complete dataset
traffic_model = train.sort_values(
    ['Junction', 'DateTime']
).copy()

# Create lag features separately for each junction
traffic_model['Lag_1'] = (
    traffic_model.groupby('Junction')['Vehicles'].shift(1)
)

traffic_model['Lag_2'] = (
    traffic_model.groupby('Junction')['Vehicles'].shift(2)
)

# Rolling mean of previous 3 observations
traffic_model['Rolling_Mean_3'] = (
    traffic_model.groupby('Junction')['Vehicles']
    .shift(1)
    .rolling(3)
    .mean()
    .reset_index(level=0, drop=True)
)

# Remove rows where lag/rolling values are unavailable
traffic_model = traffic_model.dropna(
    subset=['Lag_1', 'Lag_2', 'Rolling_Mean_3']
).copy()

print("Dataset shape after lag features:", traffic_model.shape)

display(
    traffic_model[
        ['DateTime', 'Junction', 'Vehicles',
         'Lag_1', 'Lag_2', 'Rolling_Mean_3']
    ].head(10)
)
```

Lag and Rolling Mean Feature Generation

ID	Vehicles	
0	20170701001	72.02
2952	20170701002	25.33
5904	20170701003	53.79
8856	20170701004	13.43
1	20170701011	59.98
2953	20170701012	22.60
5905	20170701013	24.15
8857	20170701014	10.13
2	20170701021	54.04
2954	20170701022	21.12

Final traffic prediction CSV

6 Performance Test

The performance of the proposed Random Forest Regression model was evaluated using a validation dataset. Three evaluation metrics were used: Mean Absolute Error (MAE), Root Mean Squared Error (RMSE), and R^2 Score.

6.1 Test Plan/ Test Cases

The following test cases were considered to verify the functionality and performance of the system:

Test Case	Description	Expected Result
TC01	Load training and testing datasets	Datasets load successfully
TC02	Preprocess date-time values	Date-time values are converted correctly
TC03	Generate time-based features	Required features are created
TC04	Generate lag and rolling mean features	Historical features are generated correctly
TC05	Train Random Forest model	Model trains successfully
TC06	Evaluate model performance	MAE, RMSE and R^2 scores are calculated
TC07	Generate test predictions	Predictions are generated for all test records
TC08	Export predictions	CSV file is saved successfully

6.2 Test Procedure

The following procedure was followed to test the system:

1. Loaded the historical training dataset and unseen test dataset.
2. Checked and preprocessed the data.

3. Extracted date-time features and generated lag and rolling mean features.
4. Divided the training data into training and validation sets.
5. Trained the baseline Random Forest model.
6. Improved the model using additional historical features.
7. Evaluated the model using MAE, RMSE and R^2 score.
8. Generated predictions for the unseen test dataset and exported them into a CSV file.

6.3 Performance Outcome

Two Random Forest models were evaluated to compare their performance.

Evaluation Metric	Baseline Model	Improved Model
MAE	5.3520	3.1206
RMSE	9.2945	5.0047
R^2 Score	0.8834	0.9662

The improved Random Forest model achieved better performance than the baseline model. The MAE decreased from 5.3520 to 3.1206, while the RMSE decreased from 9.2945 to 5.0047. The R^2 score improved from 0.8834 to 0.9662.

The improved model generated predictions for all **11,808 records** in the unseen test dataset. The final prediction CSV contained 11,808 unique IDs with no missing or negative predicted values.

These results demonstrate that feature engineering using historical traffic information improved the model's vehicle count prediction performance.

```python
# Compare baseline and improved models

comparison = pd.DataFrame({
    "Model": ["Baseline Random Forest", "Improved Random Forest"],
    "MAE": [mae, mae_imp],
    "RMSE": [rmse, rmse_imp],
    "R2": [r2, r2_imp]
})

display(comparison.round(4))
```

	Model	MAE	RMSE	R2
0	Baseline Random Forest	5.3520	9.2945	0.8834
1	Improved Random Forest	3.1206	5.0047	0.9662

Baseline vs. Improved Model performance comparison.

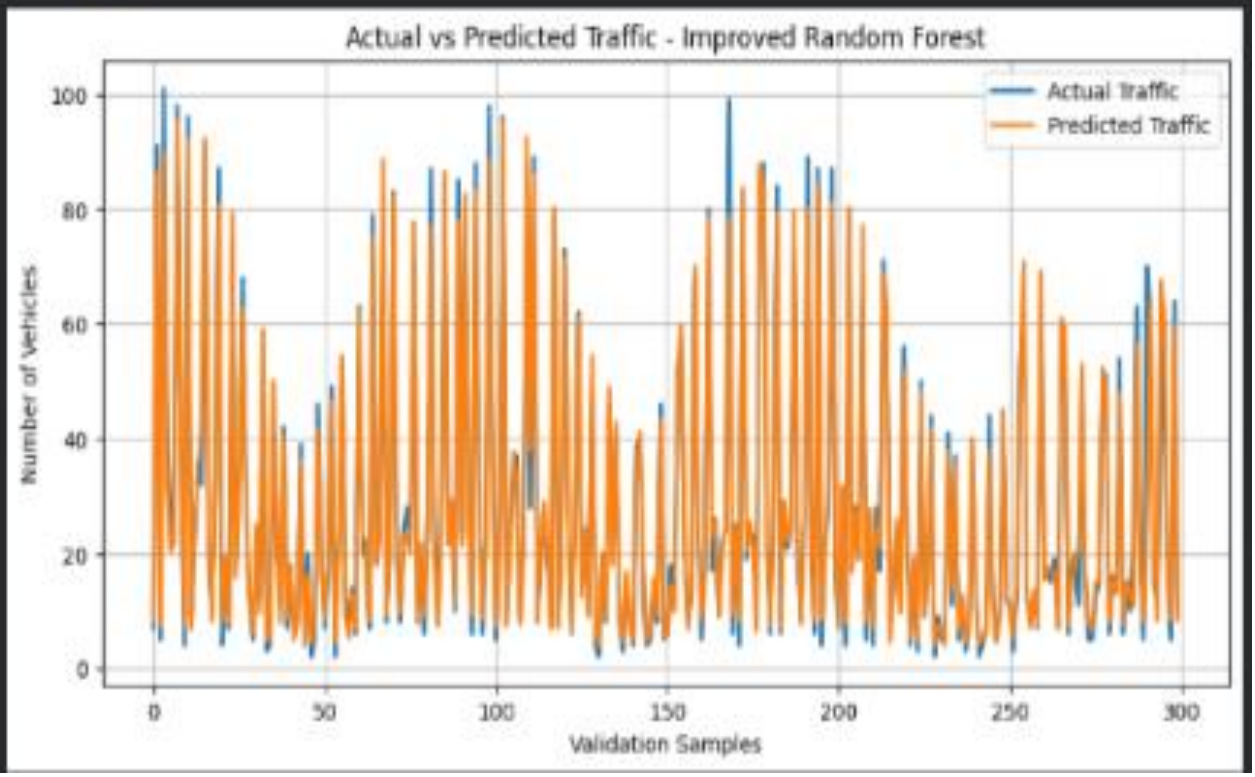
```python
import matplotlib.pyplot as plt

plt.figure(figsize=(10, 5))

plt.plot(
    y_val_imp.values[:300],
    label="Actual Traffic",
    linewidth=2
)

plt.plot(
    y_pred_imp[:300],
    label="Predicted Traffic",
    linewidth=2
)

plt.xlabel("Validation Samples")
plt.ylabel("Number of Vehicles")
plt.title("Actual vs Predicted Traffic - Improved Random Forest")
plt.legend()
plt.grid(True)
plt.show()
```



Actual vs. Predicted vehicle counts on the validation dataset.

```
# Get feature importance
importance = pd.DataFrame({
    'Feature': features,
    'Importance': rf_model.feature_importances_
})

importance = importance.sort_values(
    by='Importance',
    ascending=False
)

print("Feature Importance:")
display(importance)
```

```
Feature Importance:
```

	Feature	Importance
0	Junction	0.520463
1	Hour	0.148038
6	Year	0.127658
3	Month	0.087800
4	DayOfWeek	0.046041
2	Day	0.042242
5	IsWeekend	0.027758

Feature importance

7 My learnings

During this internship, I gained practical knowledge and hands-on experience in machine learning and data analysis through the Smart City Traffic Pattern Forecasting project.

The major learnings from this project are:

1. **Data Preprocessing:** Learned how to load, clean, and prepare real-world datasets using Pandas and NumPy.
2. **Exploratory Data Analysis:** Gained experience in analyzing traffic patterns and understanding data distributions through visualizations.
3. **Feature Engineering:** Learned to extract time-based features and create lag and rolling mean features to improve predictions.
4. **Machine Learning:** Gained practical experience in implementing and training a Random Forest Regressor.
5. **Model Evaluation:** Understood how to evaluate regression models using MAE, RMSE, and R^2 score.
6. **Performance Improvement:** Learned how suitable feature engineering can improve model performance.
7. **Project Development:** Improved problem-solving, analytical thinking, coding, and technical documentation skills.

Overall, this internship helped me understand the practical application of machine learning in solving real-world problems and strengthened my interest in data science and artificial intelligence.

8 Future work scope

The Smart City Traffic Pattern Forecasting project can be further improved and extended in several ways:

1. **Real-Time Traffic Prediction:** Integrate live traffic data from cameras and IoT sensors to predict vehicle counts in real time.
2. **Advanced Machine Learning Models:** Explore advanced algorithms such as XGBoost, LSTM, and other time-series forecasting models to improve prediction accuracy.
3. **Weather Information:** Include weather conditions such as rainfall, temperature, and visibility to understand their impact on traffic patterns.
4. **Accident and Event Detection:** Consider road accidents, construction activities, and special events that may affect traffic flow.
5. **Traffic Signal Optimization:** Use predicted vehicle counts to develop intelligent traffic signal timing systems.
6. **Web-Based Dashboard:** Develop a dashboard to display predicted traffic volume, junction-wise statistics, and traffic trends.
7. **Expansion to More Junctions:** Extend the system to cover additional traffic junctions and larger geographical areas.

These improvements can help transform the current machine learning model into a more comprehensive smart traffic management solution.